Introduction to HTML and CSS

* HTML (Hyper text markup language) is the standard markup language used to create and structure content on web pages. It defines the basic structure of a web page using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organizath of a webpage

* CSS (Cascading style sheets) is a stylesheet language language used to control the presentatn and visual apperance of HTML documents. Its used to define properties such as colours, fonts, backgrounds, spacing, spacing, borders, alignment, dimensions, and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen sizes and devices CSS can be applied using inline, internal or external stylesheets

→ HTML and CSS work together to develop webpages, where HTML provides the structures and content, while CSS provides the styling. layout, visual presentatn. The combinatn of both forms the foundatn of modern web development and enables the creatn of organized, attractive, and

user friendly web interfaces.

Difference between MERN stack and FULL stack

→ MERN stack is a Javascript based technology stack used for developing modern web applicatns. MERN stands for MongoDB, Express.js, React.js, Node.js, MongoDB is used for database managment Express.js handles the backend framework and APIs, React.js is used to develop frontend user interface and Node.js provides the server side runtime environment.

→ Full stack Development refers to the developm of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, implementing server side logic, managing databases

HTML com

1. HTML Tags -
2. Headings -
3. Paragraphs -
4. Links
5. Images
6. Tables
7. Lists
8. Forms
9. Div a
10. Sem

HTML components :-

1. HTML Tags - Define the structure and elements of a web page
2. Headings - used to display titles and headings using <h1> to <h6>
3. Paragraphs - used to display text using the <p> tag
4. Links - used to navigate between webpages using <a> tag
5. Images - used to display images using <image> tag.
6. Tables - used to organize data in rows and columns
7. Lists - used to create ordered and unordered list using <ol>, <ul>,
8. Forms - used to collect user i/p using elements such as text boxes, buttons and check boxes
9. Div and Span - used to group and organize webpage content.
10. Semantic elements - elements such as <header>, <nav>, <section>, <article>, <footer>, Provide meaningful webpage structure.

CSS components :

1. Selectors - select HTML elements to apply styles.
2. properties - define visual characteristics of elements
3. values - specify the setting of a css property
4. colors - define text, background, border colour.
5. Fonts - control font family, size style and weight
6. Backgrounds - set background colour and images.
7. Borders - define the border style, width and radius.
8. Margin - control the space outside an element
9. Padding - controls the space inside element
10. layout - controls the arrangment of elements using flexbox, grid and positioning.

HTML vs HTML5

HTML	HTML 5
old version of HTML	latest major release of HTML
limited multimedia support	supports audio and video
uses external plugins for multimedias	no external plugs needed
has fewer semantic tags	provides semantic tags.
no <canvas> element	support <canvas>
limited web storage	supports local storage
complex DOCTYPE declaration	simple <!DOCTYPE HTML

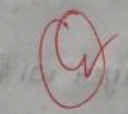
CSS vs CSS3

CSS	CSS3
Earlier version of CSS	Advanced version of CSS
basic styling features	provides advanced styling features
limited animations	supports animations and transitions
no flexible layout systems	supports flexbox and grid
limited visual effects	supports shadows, gradients effects
no media queries	supports animations
use a single specification	divides into different modules.

Design a web page using HTML and CSS static pomodoro timer, using HTML & CSS:

```
<!DOCTYPE HTML>
<html>
<head>
  <title pomodoro timer</title>
  <style>
    body {
      font-family: Arial;
      background: #555555;
      text-align: center;
      padding: 50px
    }
    .timer {
      width: 300px;
      margin: auto;
      padding: 30px;
      background: white;
      border-radius: 15px;
      box-shadow: 0 0 10px gray;
    }
    .time {
      font-size: 50px;
      margin: 20px;
    }
    button {
      padding: 10px 20px;
      margin: 5px;
      border: none;
      border-radius: 5px;
      background: 5px;
      background: #E74C3C;
```

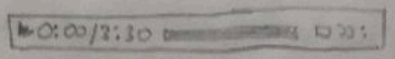
```
colour: white;
}
</style>
</head>
<body>
    <div class="timer">
        <h1>pomodoro timer</h1>
        <div class="time">25:00</div>
        <button>start</button>
        <button>Reset</button>
    </div>
</body>
</html>
```



code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Audio Example</title>
</head>
<body>
  <h2>my fav song</h2>
  <audio controls>
    <source src="song.mp3" type=
        "audio/mpeg">
    your browser doesnt support audio tag
  </audio>
</body>
</html>
```

o/p:- Displays an audio player with play, pause and volume controls.



steps:-

Create a new HTML file

Add the basic HTML structure

Add heading using <h2> tag

Insert the <audio> tags with control attribute

Specify the audio file using <source> tag.

Save and open the HTML file in a web browser

Play the audio using displayed controls

code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Video Example</title>
</head>
<body>
    <h2>My video</h2>
    <video width="400" controls>
        <source src="video.mp4" type="video/mp4">
    </video>
</body>
</html>
```

Opt: displays a video player with play, pause, volume, full screen controls.

Steps:

Create a new HTML file

Add a basic HTML structure

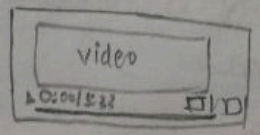
Add a heading using the <h2> tag

Insert the <video> tag with control attributes

Specify the video file using <source> tag

Save the file and open it in web browser

Play the video using displayed controls.



Write a program using Javascript (arrays, functions)

Code:-

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11]
function displayArray(arr) {
    console.log("Array Elements:");
    for (let i=0; i<arr.length; i++){
        console.log(arr[i]);
    }
}
function findSum(arr) {
    let sum = 0;
    for (let i=0; i<arr.length; i++){
        sum += arr[i];
    }
    return sum;
}
```

4. Develop
Program

Experiment-4

4. Develop dynamic webpage using JavaScript program:

```
<!DOCTYPE html>
<html>
<head>
  <title> Dynamic webpage </title>
  <style>
    body{
      text-align: center;
      font-family: Arial;
      margin-top: 100px;
    }
    button{
      padding: 10px 20px;
      font-size: 14px;
      cursor: pointer;
    }
  </style>
</head>
<body>
  <h1 id="heading">welcome to
    my webpage </h1>
  <p id="message"> click the button
    to change this webpage </p>
  <button click="change"> click
    button
```

```
<script>function changepage(){
        document.getElementById
        ("heading").
innerHTML = "welcome to javascript";
document.getElementById("heading")
    innerHTML = "The webpage has been
        changed dynamically";
document.body.style.background-color
    = "lighter"
}
</script>
</body>
</html>
```

O/P:

welcome to my webpage
click the button to change this
webpage

[click me]

* when the user clicks "click me", the javascript function changepage() is called, then the heading is dynamically changes and background also changes.

clicking the button:

welcome to javascript
The webpage has been changed dynamicaly

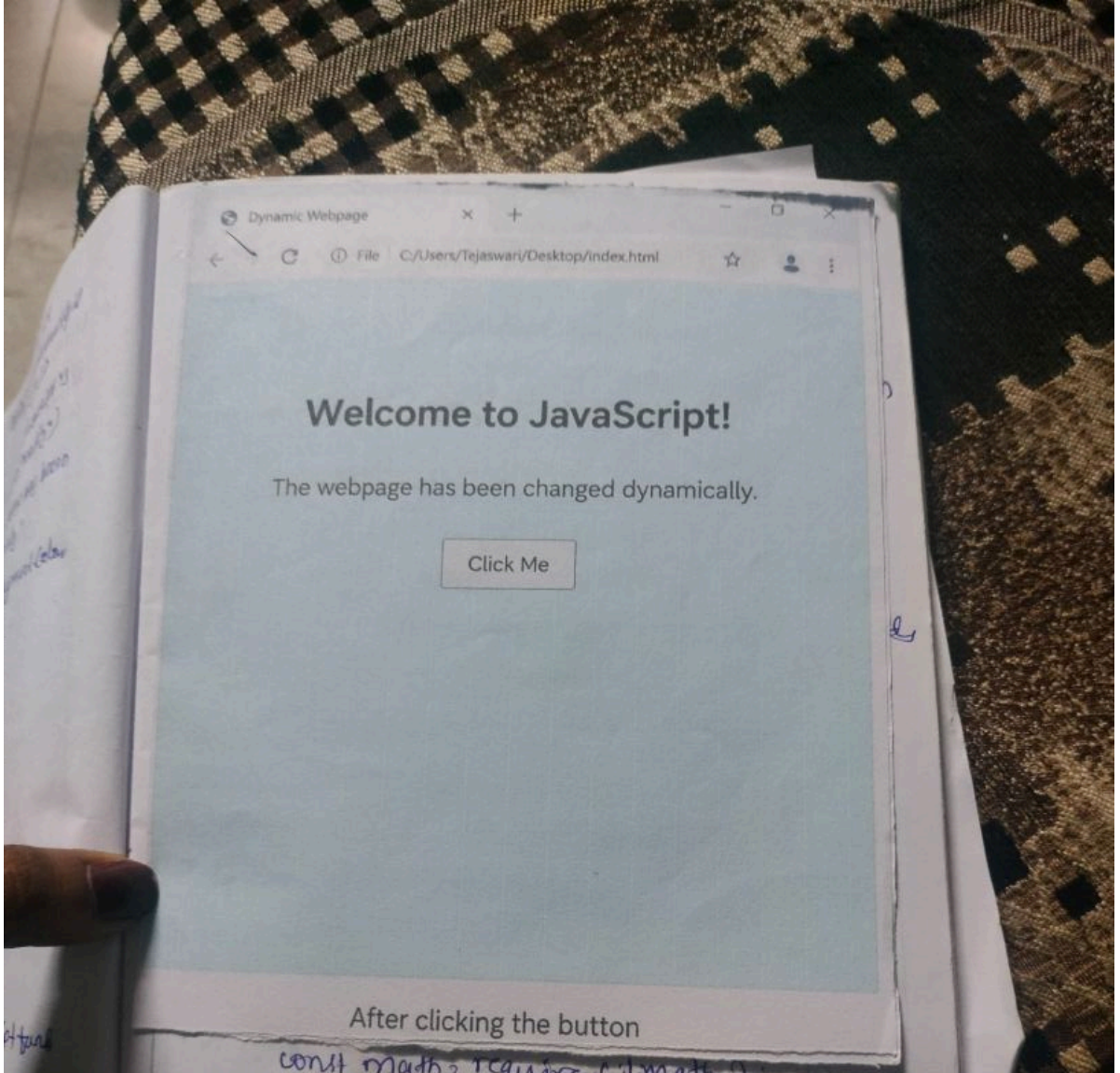
[click me]

Dynamic Webpage

W

The

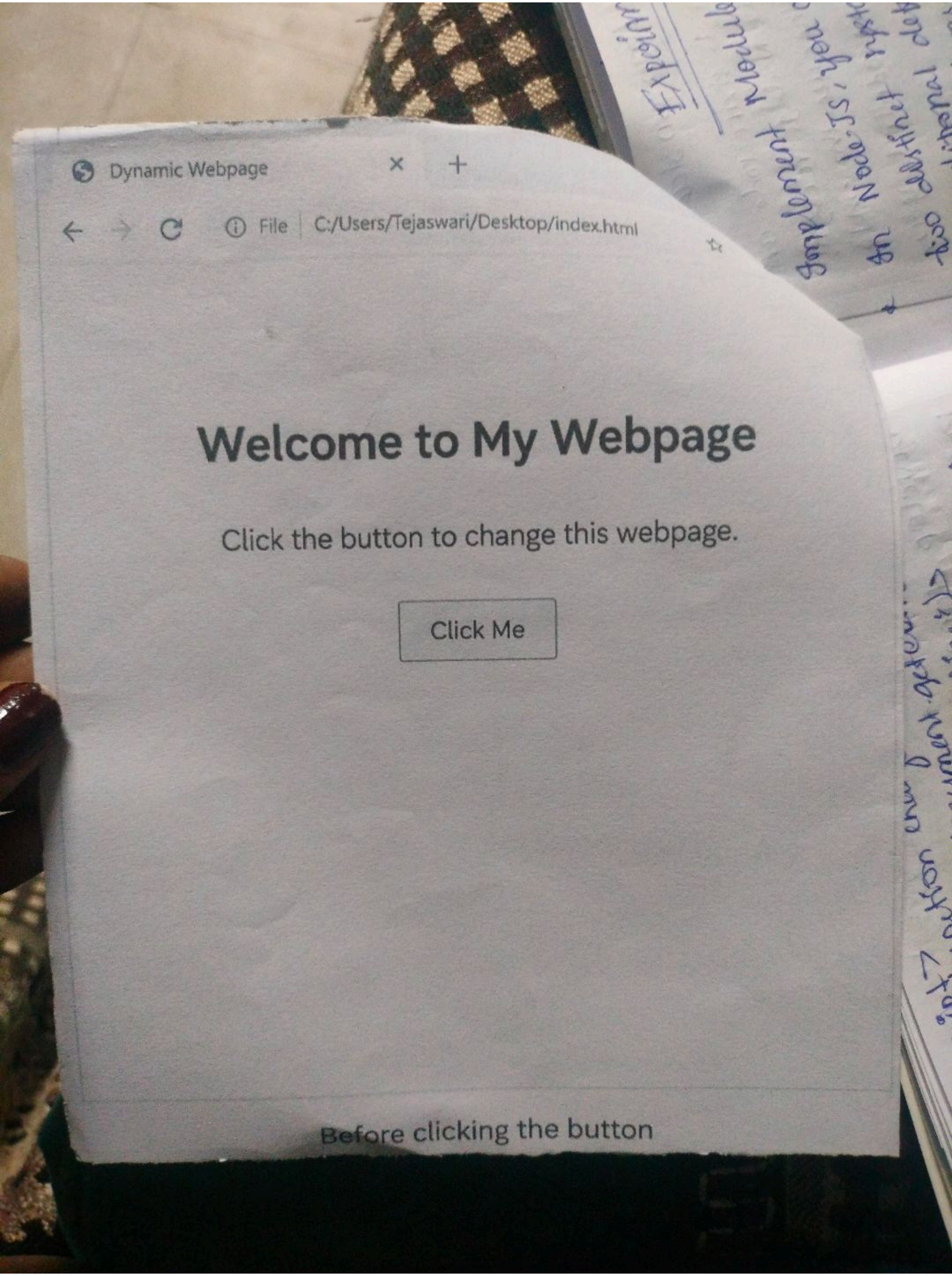
O/P



After clicking the button

```
const math = require('./math');
console.log(math.add(10,5));
console.log(math.subtract(10,5));
```

O/P: 15
5


Dynamic Webpage
File
C:/Users/Tejaswari/Desktop/index.html
Welcome to My Webpage
Click the button to change this webpage.
Click Me
Before clicking the button

Experiment-05

Implement Modules in Node JS

In Node.JS, you can implement module, using two distinct systems common JS (CJS), which is traditional default, and ES modules (ESM), the modern Javascript standard.

1. The common JS method (default):

common JS uses module.exports to share code and require() to import it. This system loads modules synchronously.

a. create the module (math.js):

```
const add = (a,b) => a+b;
const subtract = (a,b) => a-b;
module.exports = { add,
                   subtract
                 };
```

b. use the module (app.js):

```
const math = require('./math');
console.log(math.add(10,5));
console.log(math.subtract(10,5));
```

O/P: 15
5

2 The ES module method (modern standard)

ES modules use export to share code and import to use it. To make Node.js recognize ES modules, you must, either use the .mjs file extension or add "type": "module" to your package.json file.

a) create the module (calculator.js):

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
const calculator name = "supercalc"
export default calculator Name
```

b) use the module (main.js)

```
import calculator Name, { multiply,
                          divide } .
                  from './calculator.js';
console.log (calculator name);
console.log (multiply (4,2));
console.log (divide (4,2));
```

o/p:- supercalc.

8

2

3. Using Built in core modules:

Node.js also comes equipped with dozens of built-in core modules (tickets for file system or path to file reality) that don't need install

we wi

Ex.

we will load them by module names!

- common JS style

```
const fs = require('fs')
```

// Es module style

```
import fs from 'fs';
```

Here are some of the most commonly used ones!

1. fs = file system operations
2. http = HTTP server and client
3. path = file path utilities.
4. os - operating system utilities
5. events - event handling.
6. util - utility functions
7. stream - stream handling
8. crypto - cryptographic functions.
9. url - URL parsing
10. querystring - URL query string handling

Ex. Simple HTTP server

```
http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type':
                        'text/html'});
    res.end("Hello world!");
}).listen(8080);
```

o/p, Helloworld!

Experiment-06

06. Develop mini application using express and node.JS

- simple mini application using node JS + express JS

Student management system

1. Add a student
2. View all students
3. delete a student
4. uses Express.js routes
5. simple HTML interface

Project structure

```
student-app1
  ├ server.js
  └ public
      └ index.html
```

server.js

```
const express = require('express');
const app = express();
const PORT = 8000;
let students = [];
app.use(express.json());
app.use(express.static('public'));
```

```
app.get("/students",(req,res)=>{
    res.json(students);
});

//Add
app.post("/students",(req,res)=>{
    const student={
        id: students.length+1,
        name:req.body.name,
        course:req.body.course,
    };
    students.push(student);
    res.json({message:"student added successfully",
        student:student});
});

//Delete
app.Delete("/students/:id",(req,res)=>{ const
    id=parseInt(req.params.id);
    students=students.filter(student=>
    student.id!==id);
    res.json({
        message:"student deleted successfully"
    });
});

// start server
```

```
app.listen(port, () => {
  console.log("server running at http://
      localhost:${port}`);
});
```

public/index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title> student Managment </title>
</head>
<body>
  <h1>student managment system </h1>
  <input type="text" id="name"
  placeholder="Enter student
                name">
  <input type="text" id="course"
  placeholder="Enter course">
  <button onclick="addstudent()">
  <addstudent</button>
  <h1> Student list </h1>
  <ul id="studentlist"></ul>
<script>
  function addstudent(){
```

```
let name = document.getElementById
            ("name").value;
let course = document.getElementById
            ("course").value;

fetch("/students", {
        method: "POST",
        headers: {
            "Content-Type":
                "application/json"},
        body: JSON.stringify({
                name: name,
                course: course
            })
        })
    .then(response => response
            .json())
    .then(data => {
        alert(data.message());
document.getElementById("name").value = "";
document.getElementById("course").value = "";
        displayStudents();
    });
}

function displayStudents() {
    fetch("/students").then(response =>
            response.json())
```

```
.then(students) => {
let list = document.getElementById("studentList");
list.innerHTML = "";
students.forEach(student => {
let li = document.createElement("li");
li.innerHTML = student.id + ": " +
student.name + " - " +
student.course + "
<button onclick="deleteStudent(' + student.id + ')">
Delete
</button>";
list.appendChild(li);
});
});
}
function deleteStudent(id) {
fetch("/students/" + id, { method:
"DELETE" })
.then(response => response.json())
.then(data => {
alert(data.message);
displayStudents();
});
}
```

3

displayStudents():

</script>
</body>
</html>

op:

Student management System

[Enter student name]

[Enter code]

[Add student]

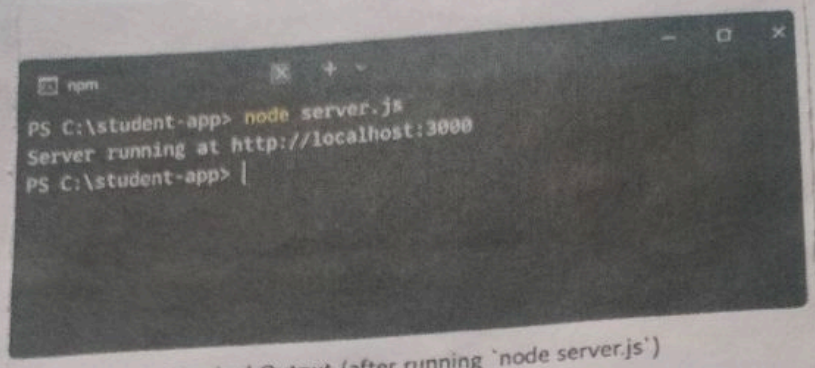
student-list:

1. Harshitha - CSM [delete]
2. Harshith - CSM [delete]

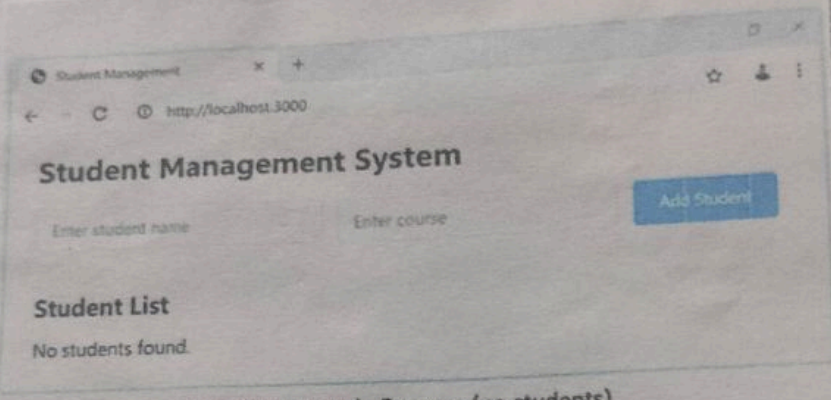
The express.js server is successfully started at

http://localhost:3000

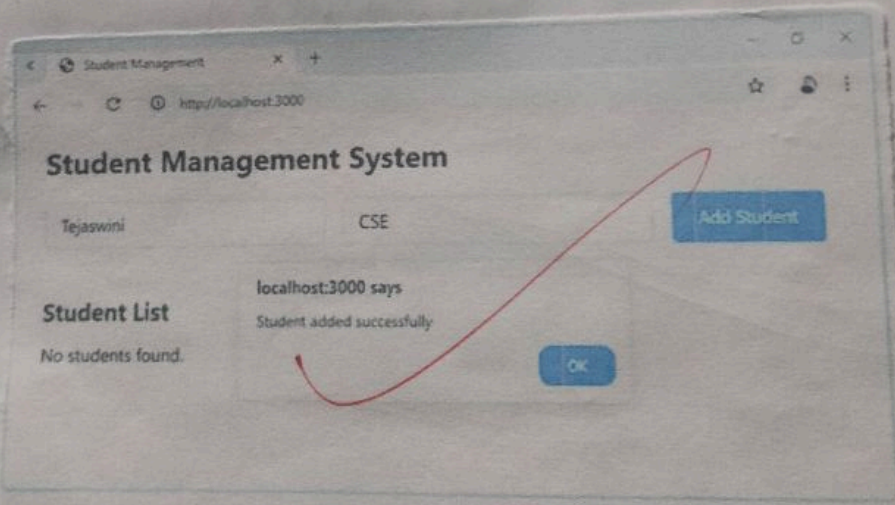
The browser display the student managment system with fields



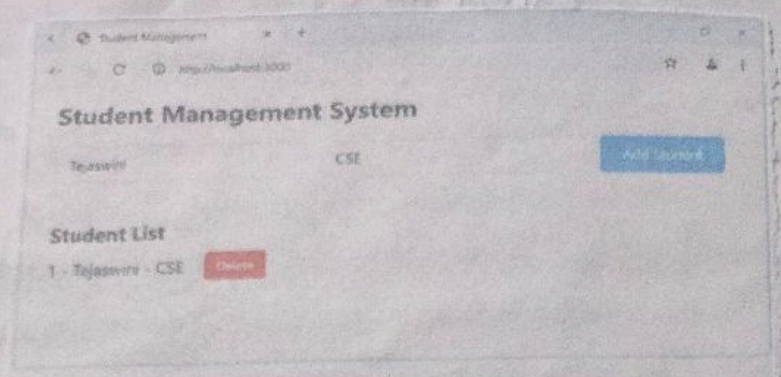
1. Terminal Output (after running `node server.js`)



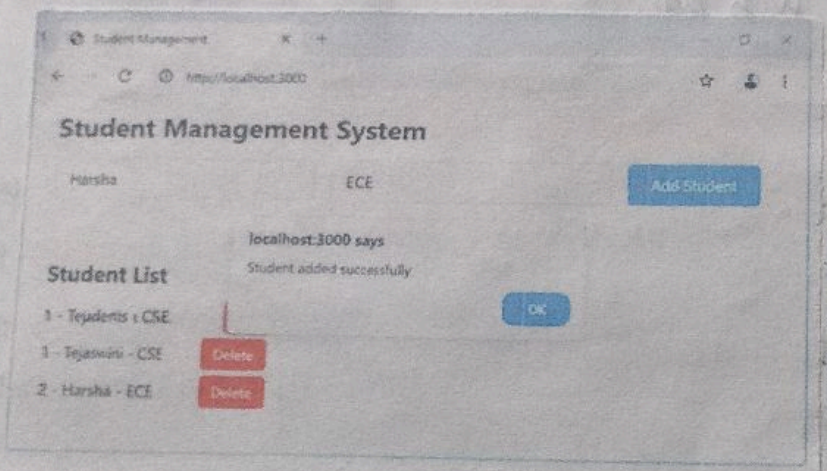
2. Initial Output in Browser (no students)



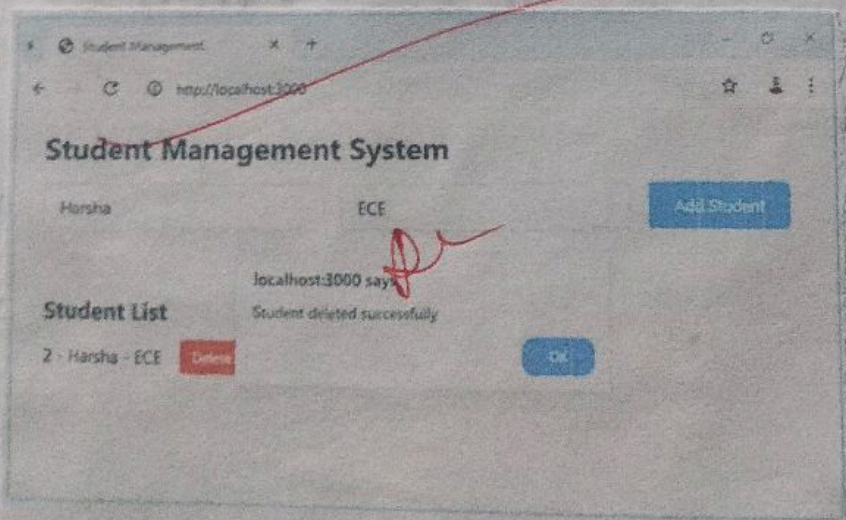
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment-07

7) Create a functional components and class components using react

AIM: To develop a react application using functional components and class components to display students details, manage state, handle button click and dynamically update data on a web page.

Funitional Component:

A funitional component is a javascript function that returns JSX to display the user interface. It uses React hooks such as useState to manage state and update the webpage dynamically.

Program:

```
import React, {useState} from
'react';
function StudentFunctional(){
    const [name, setName] = useState('Harshi');
    const [course, setCourse] = useState('CSM');
    const [count, setCount] = useState(0);
    const changeName = () => {
        setName('Harshithe');
    };
```

```
const changeCourse = () => {
    setCourse("CSE(AI&ML)");
};
const incrementCount = () => {
    setCount(count+1);
};
return (
    <div style={{
        border: "2px solid blue"
        padding: '20px',
        margin: '20px',
        borderRadio: '10px',
        textAlign: 'center',
    }}>
    <h1>Student Details</h1>
    <h2> Functional Component </h2>
    <p> <b>Name:</b> {name} </p>
    <p> <b> course: </b> {course} </p>
    <p> <b> button click: </b> {count} </p>
        ChangeName </button>
    <button onClick={changeCourse}>
        ChangeCourse </button>
```

<button onclick={() => increment(count)}>Increment
- cnt </button>
);}

output: when the user clicks the buttons the names, course and count change dynamically without reloading the webpage.

• Class

A

class that

the renele

manages

Program

impe

're

cla

• Class component

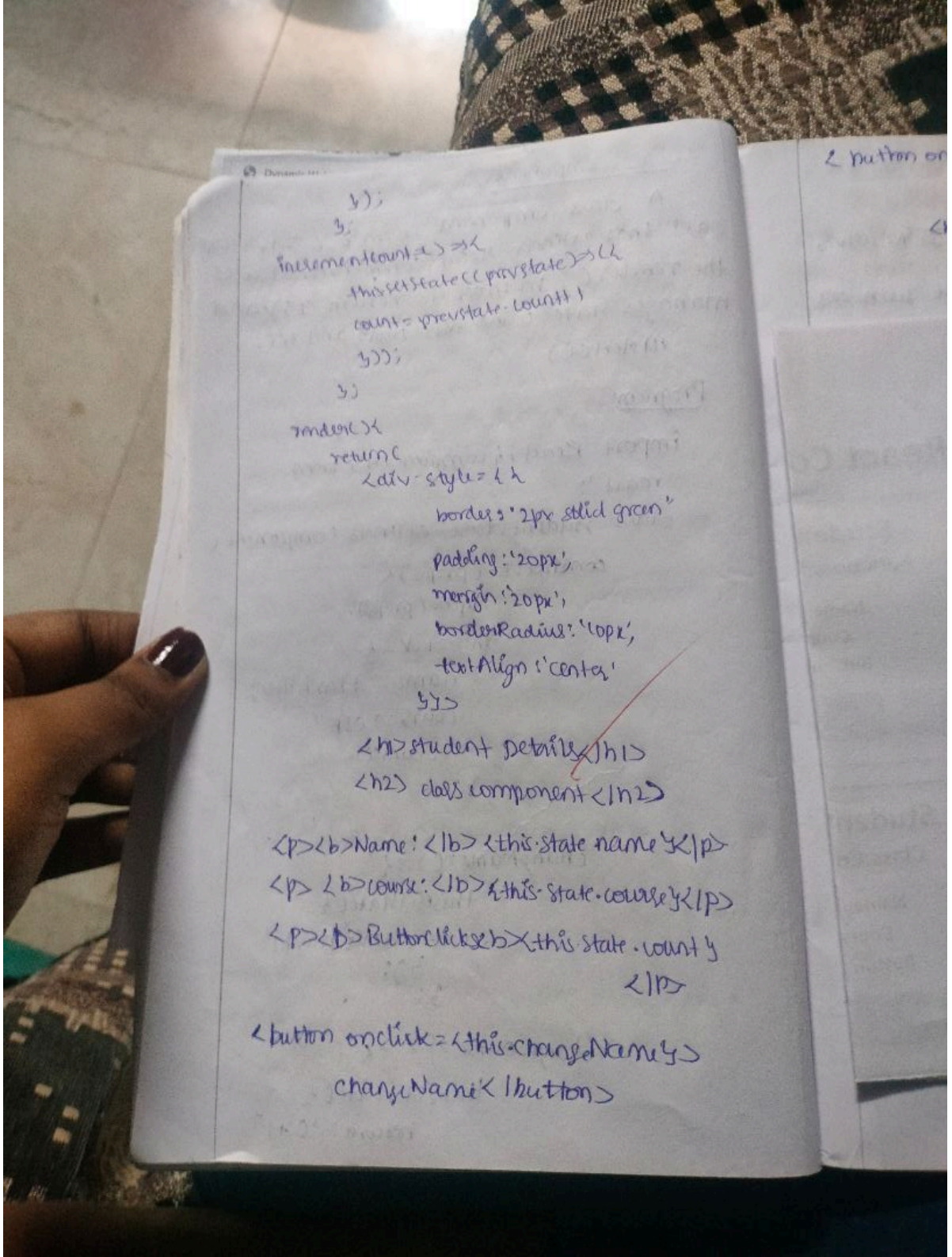
A class component is an ES6 JavaScript class that extends react components. It uses the render() method to return JSX and manages state using this.state and this.setState()

Program:-

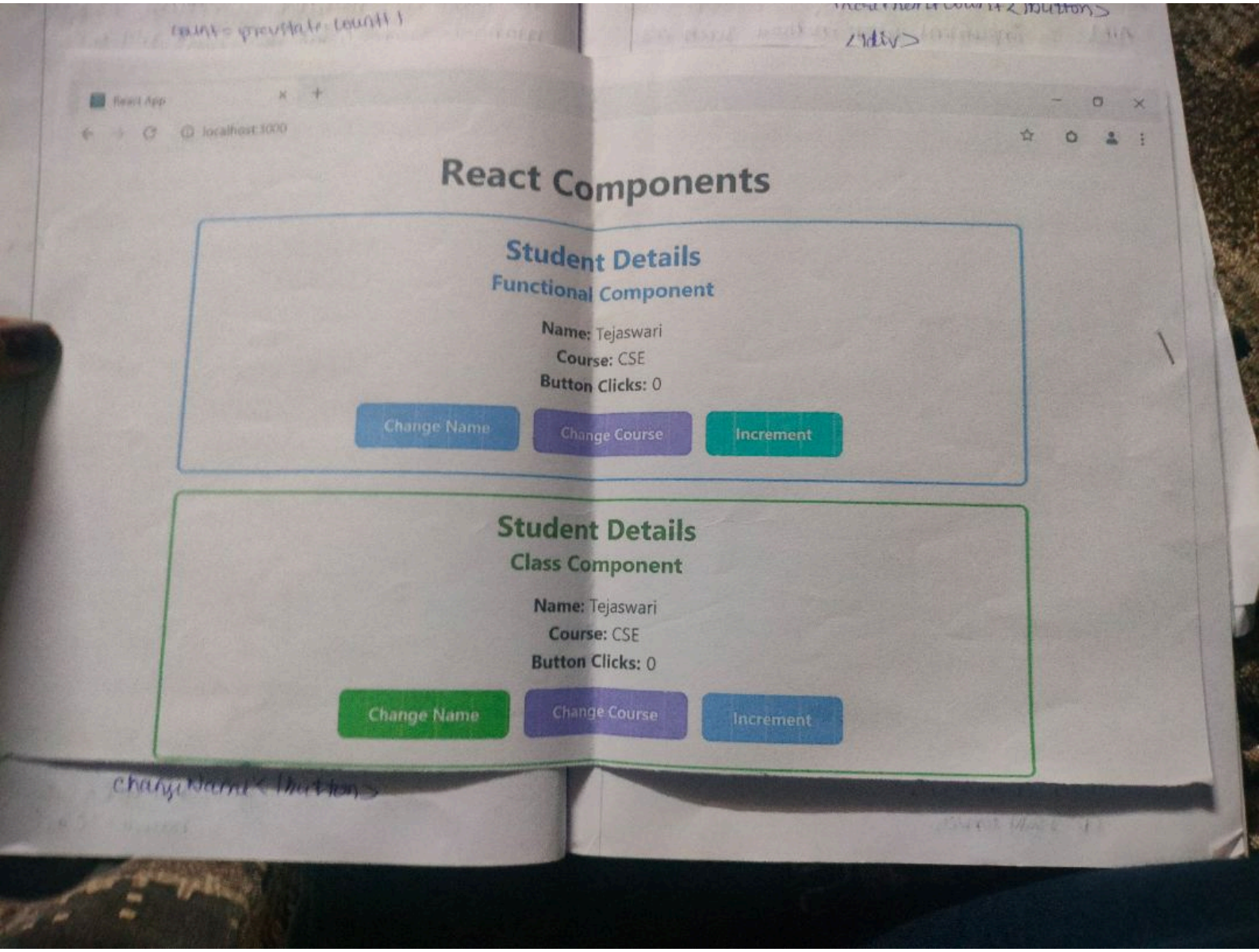
```
import React, {Component} from 'react';
class StudentClass extends Component {
    constructor(props) {
        super(props);
        this.state = {
            name: "Harshitha",
            course: 'CSE',
            count:0
        };

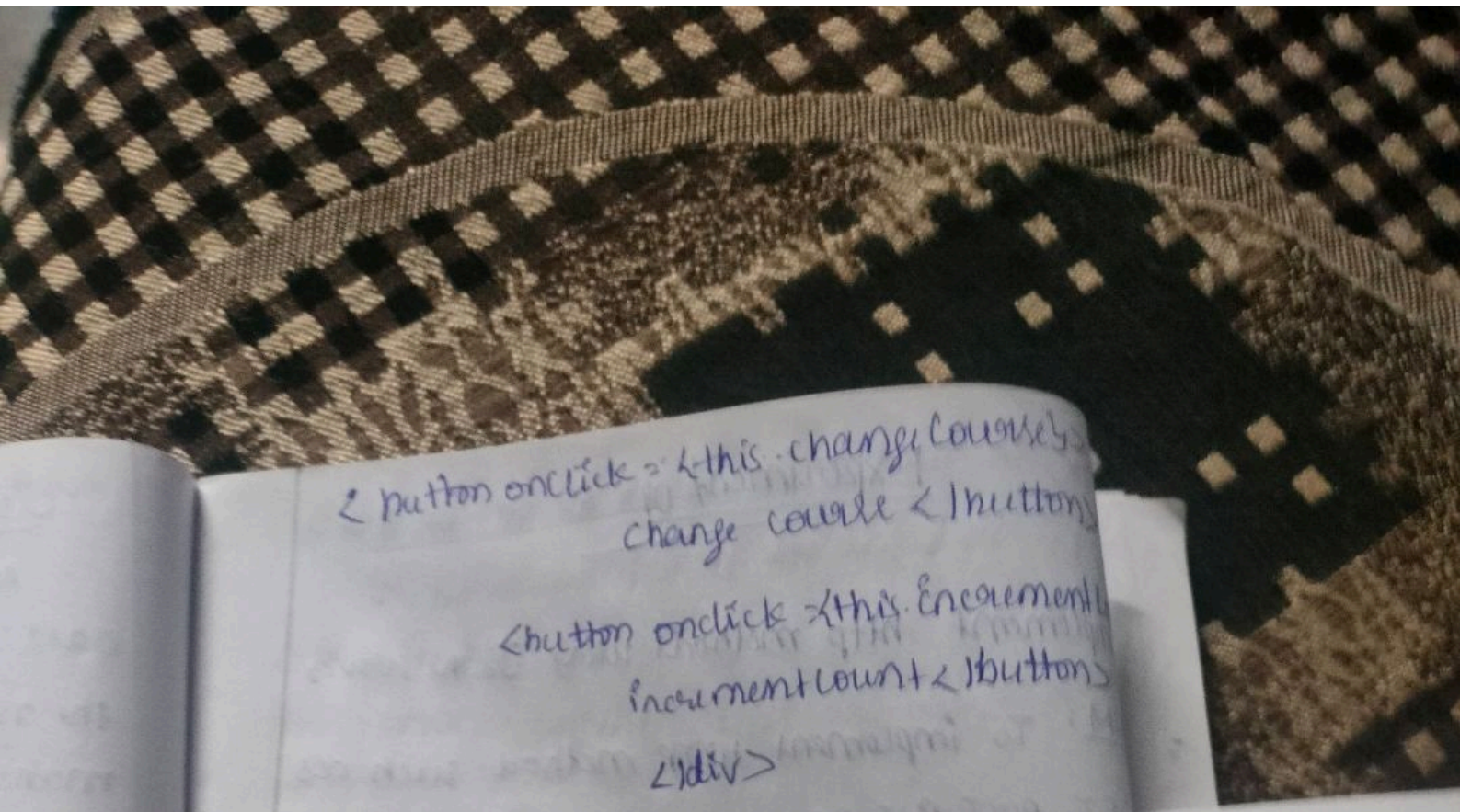
    changeName = () => {
        this.setState({
            name: 'Harshi'
        });
    }

    ChangeCourse = () => {
        this.setState({
            course: 'CSM'
```



```
});
};
incrementCount = () => {
  this.setState((prevState) => {
  count = prevState.count + 1
  });
});
};
render() {
  return (
    <div style={{
        border: "2px solid green"
        padding: '20px',
        margin: '20px',
        borderRadius: '10px',
        textAlign: 'center'
      }}>
      <h1>Student Details</h1>
      <h2> class component </h2>
<p><b>Name: </b> {this.state.name}</p>
<p> <b>course: </b> {this.state.course}</p>
<p><b> Button click</b> {this.state.count}
                                </p>
<button onClick={this.changeName}>
    changeName</button>
```


React App
localhost:3000
React Components
Student Details
Functional Component
Name: Tejaswari
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment
Student Details
Class Component
Name: Tejaswari
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment



```
<button onClick={this.changeCourse}>
    Change course</button>

<button onClick={this.IncrementC}>
    Increment count</button>

</div>
```

O/P:

Dynamic Webpage

Experiment-09

8) Implement http methods using JSON server

AIM: To implement HTTP method such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD (create, read, update, delete) operations student records

THEORY: JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

HTTP methods are used to communicate b/w a client and a server

- GET - Retrives data from server
- POST - Adds new data to the server
- PUT - Replace an existing record.
- PATCH - Updates selected fields of a record.
- DELETE - Removes a record from the server

a. Create a JSON server:

- Step-1: Initialize the project.

Open the VS code terminal and execute the following commands:

```
mkdir JSON Server
CD JSON server
```

npm init -y
npm install json server

Step-2) create db.json

create a file named dbjson and add the following students data.

```
{ "students": [
  { "id": "1";
    "name": "Harshitha",
    "course": "CSM";
    "marks": 90 },
  { "id": "2",
    "name": "Harshi" };
    "course": "CSM";
    "marks": 90 },
  { "id": "3"
    "name": "Alex",
    "course": "CSE";
    "Marks": 91
  }
```

Step-2 : start the JSON server

for the current JSON server version execute

npx json-server db.json

The server starts at :

http://localhost:3000

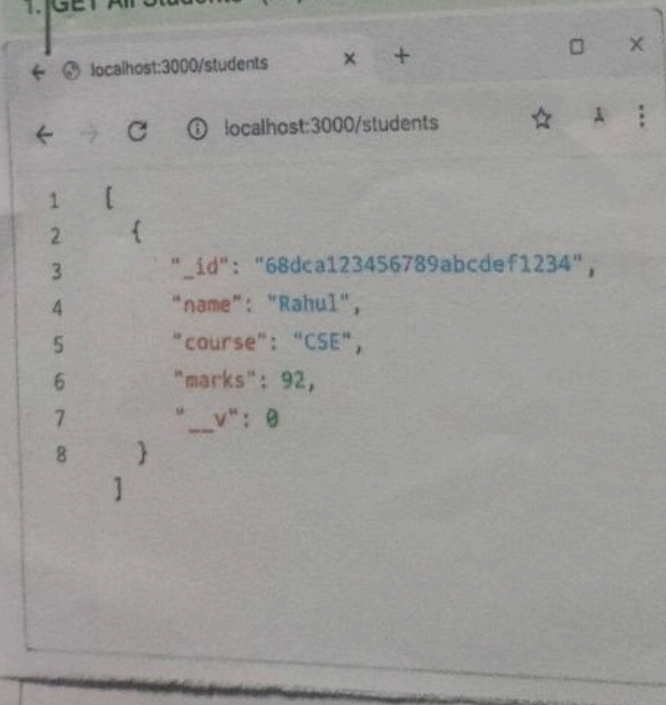
Implementation of HTTP methods

- GET method: The get method is used to retrive student records from JSON server

code:-

```
fetch('http://localhost:3000/students')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(error => console.log(error));
```

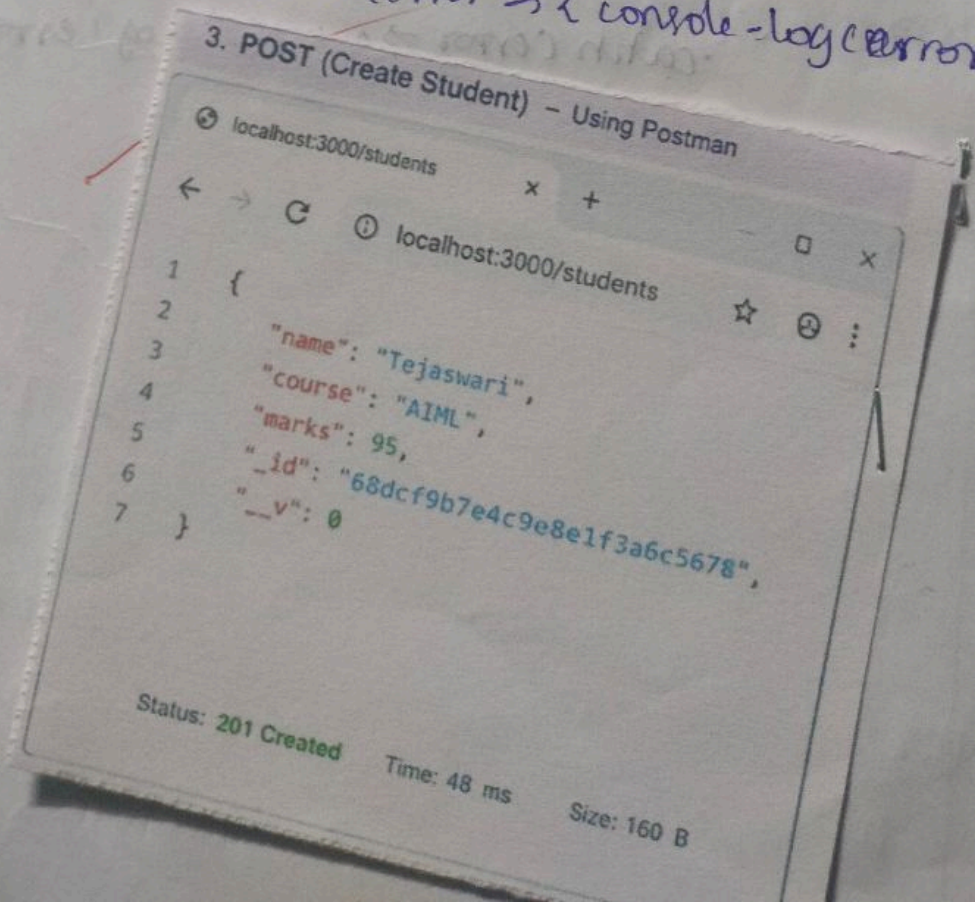
1. GET All Students (http://localhost:3000/students)



B) POST method: The POST method is used to insert a new student record in server

code:

```
fetch ('http://localhost:3000/students',{
    method: 'POST',
    headers:{
        'content-Type': 'application/json',
    },
    body: JSON.stringify ({
        name: 'Rahul',
        course: 'CSE',
        marks: 92
    })
})
.then (response => response.json())
.then (data => { console.log (data);
    })
.catch (error => { console.log (error); });
```



c) PUT method :- The PUT method is used to re
on existing student records with updated d
code:- fetch('http://localhost:3000/students/
{ method: 'PUT',
headers: {
'content-Type': 'application/json'
},
body: JSON.stringify({
name: 'Rahul',
course: 'AIML',
marks: 96,
})
})
.then(response => response.json())
.then(data => {
console.log(data); })
.catch(error => console.log('error'));})

D> PATCH
to upda
records
code:-

4. PUT (Update Student) – Using Postman

localhost:3000/students

localhost:3000/students/68dca123456789abcdef1234


```
{
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "AIML",
    "marks": 96,
    "__v": 0
}
```

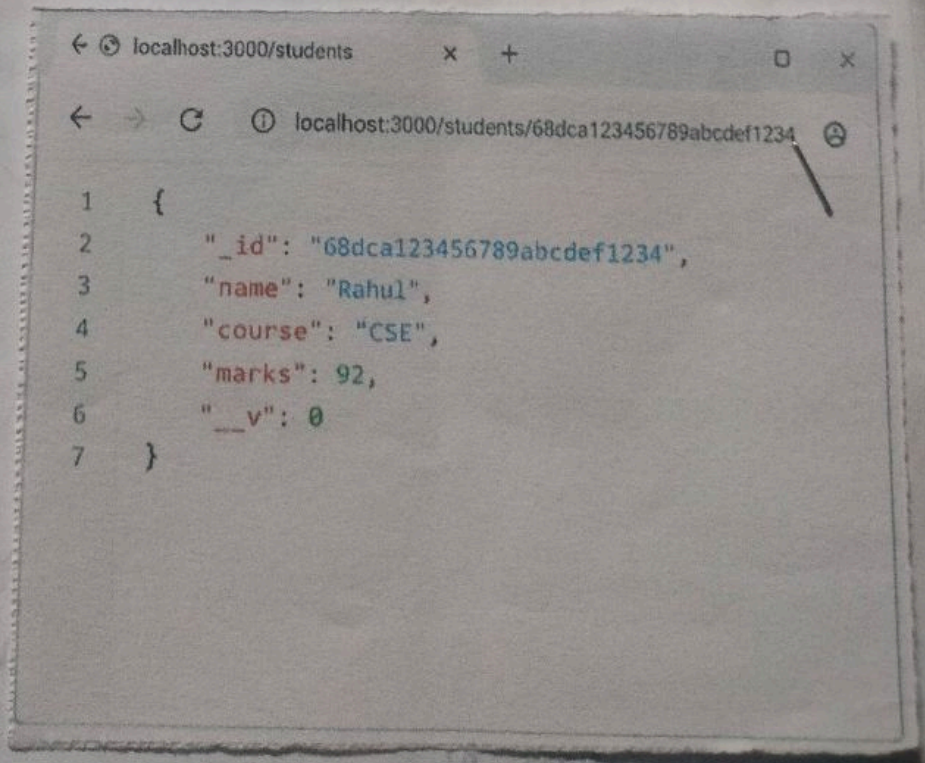

Status: 200 OK Time: 62 ms Size: 165 B

D) PATCH Method: The patch method is used to update only selected fields of an existing students records

Code:

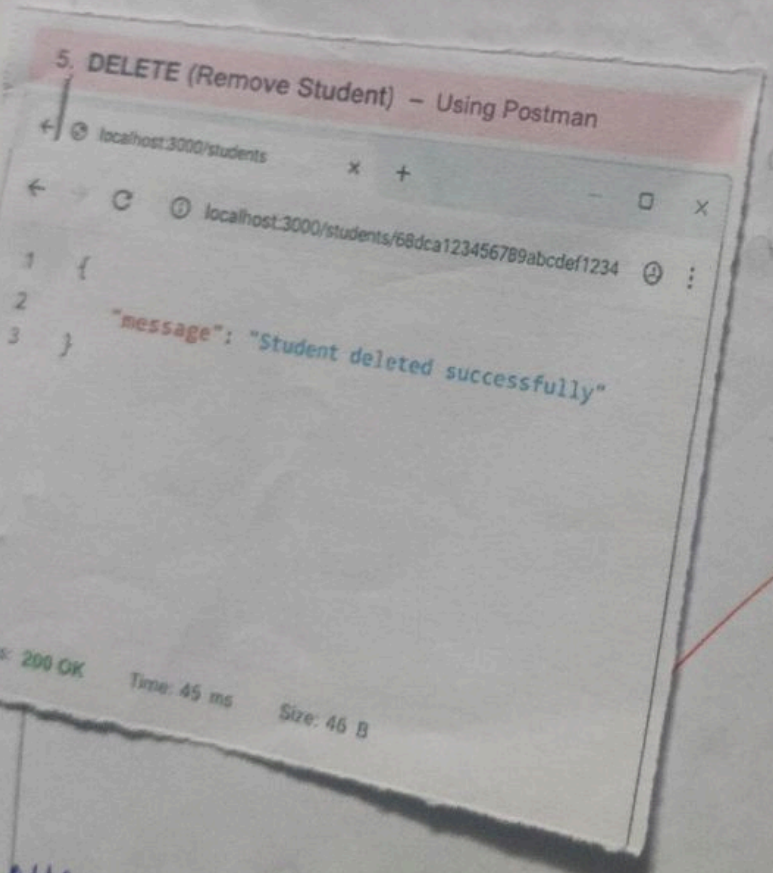
```
fetch('http://localhost:3000/students/',{
  method: 'PATCH',
  header: {
    'content-Type': 'application/json',
  },
  body: JSON.stringify({
    marks:96
  })
})
.then(response => response.json())
.then(data => {console.log(data);})
.catch(error => {console.log(error);})
```

Output



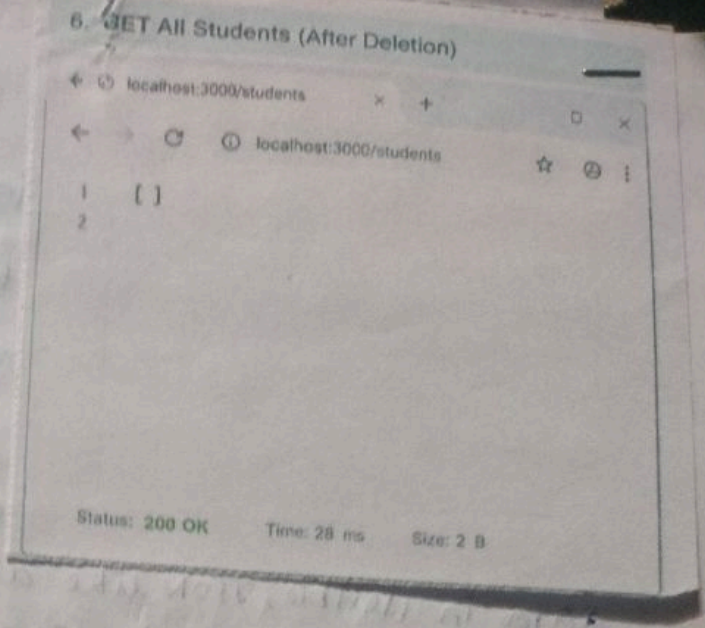
E) DELETE method: The Delete method is used to remove student records from server.

```
code: fetch("https://localhost:3000/students/3", {
        method: 'DELETE'
      })
      .then (response => { if (response.ok) {
        console.log ("student deleted
                      successfully");
      }})
      .catch(error => console.log(error));
```



- After executing the POST, PUT, PATCH, and DELETE operations in sequence the remaining students will be.

6. GET All Students (After Deletion)



Thus HTTP methods GET, POST, PUT, PATCH, DELETE were successfully implemented using JSON servers. The CRUD operations were performed on student records. The data was retrived, inserted updated and deleted using HTTP request

Experiment - 9

9) Develop CRUD operations using MongoDB

Aim: To develop a webapplication using Node.js, express.js and mongoDB to perform CRUD on student records

THEORY: MongoDB is a NoSQL database that stores data in flexible, JSON like documents. CRUD operations are used to manage data in database.

Mongoose is an object oriented modeling tool. Library for MongoDB and Node.js. It helps define schemas, interact with collections and perform database operations.

1. Installation and Project setup:

```
mkdir MongoDB-CRUD
cd MongoDB-CRUD
npm init -y
npm install express mongoose
```

2. Program: server.js

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
const PORT = 3000;
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1:27017/college')
.then(()=>console.log("MongoDB connected"))
.catch(err => console.log(err));

// create student
const studentSchema = new
  mongoose.Schema({
    name: {
      type: 'string',
      required: true
    },
    course: {
      type: string,
      required: true
      },
    marks: {
      type: Number,
      required: true
    }
    });

// Create student model
```

create student :
mongoose.model('Student', studentSchema);

//Create : Add
app.post('/students', async (req, res) =>{
try{
const student = new student(req.body);
const saved student = await
student.save();
res.status(201).json(savedStudent); }
catch (error){
res.status(400).json({ message:
error.message}); }
});

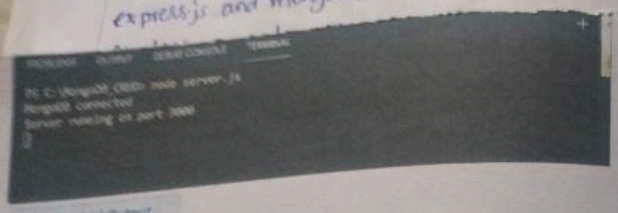
//Read : Get All students

Experiment-9

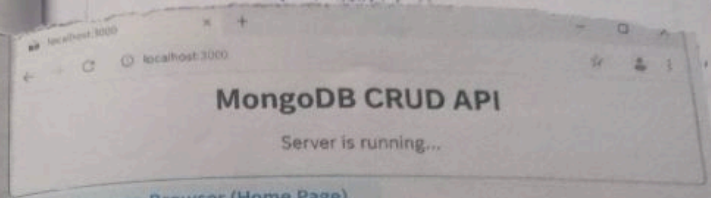
a) Develop CRUD operations using MongoDB

Aim: To develop a web application using Node.js, express.js and mongoDB to perform CRUD on

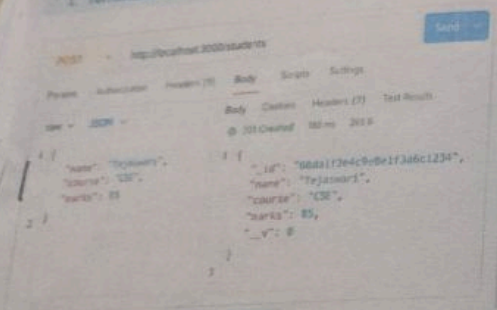
```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
const PORT = 3000;
app.use(express.json());
```



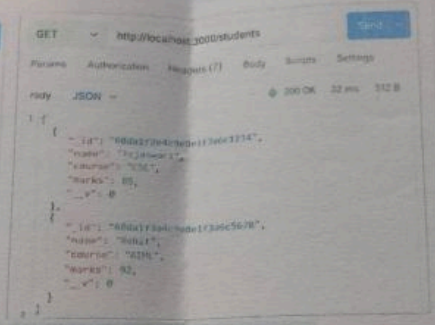
1. Terminal Output



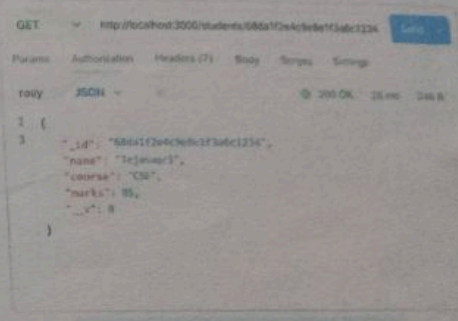
2. Chrome Browser (Home Page)



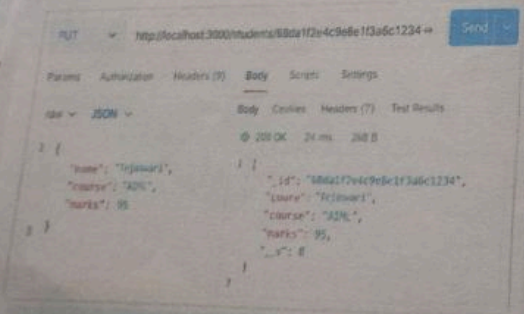
3. Create (POST) – Add Student



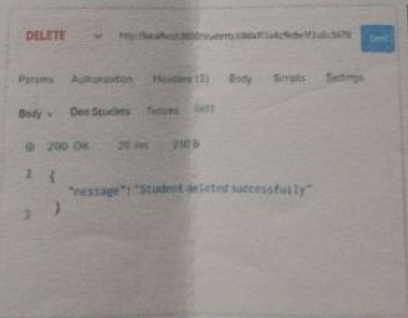
4. Read (GET) – Get All Students



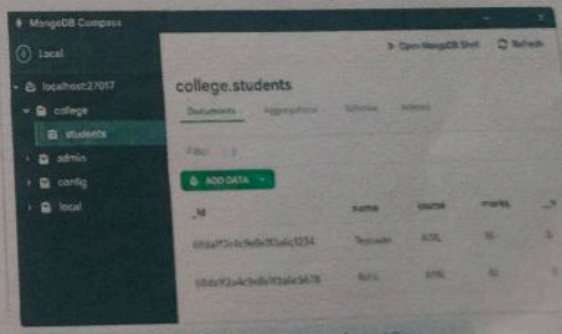
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View